

Hadoop YARN Server

核心架构设计文档

基于 release-3.3.5-RC0 版本源码深度分析

项目名称	Apache Hadoop YARN Server
源码版本	release-3.3.5-RC0
文档类型	核心架构设计文档
模块范围	hadoop-yarn-server (14个子模块)
核心文件数	1165+ Java 源文件
生成日期	2026年3月24日

目 录

1. 项目概述与模块总览
2. 整体架构设计
3. ResourceManager 核心架构
4. NodeManager 核心架构
5. 状态机设计模式详解
6. 调度器体系架构
7. Federation 联邦架构
8. 核心流程 — 应用提交与执行
9. 核心流程 — 容器生命周期管理
10. 核心流程 — 节点心跳与资源更新
11. 核心流程 — 调度与资源分配
12. 关键设计模式总结
13. 关键场景调用链分析
14. 接口与扩展点
15. 总结

1. 项目概述与模块总览

Hadoop YARN (Yet Another Resource Negotiator) 是 Hadoop 生态系统中的资源管理和任务调度框架。hadoop-yarn-server 是 YARN 的服务端核心实现，包含了 ResourceManager、NodeManager、Router 等关键服务组件。本文档基于 release-3.3.5-RC0 版本源码，深入分析其架构设计、核心流程和关键设计模式。

1.1 子模块总览

hadoop-yarn-server 由 14 个子模块组成，每个模块承担不同的职责。以下表格展示了各模块的核心定位和规模：

模块名称	简称	文件数	核心职责
hadoop-yarn-server-resourcemanager	ResourceManager	500+	集群资源管理和调度核心
hadoop-yarn-server-nodemanager	NodeManager	402	节点级容器生命周期管理
hadoop-yarn-server-common	Common	348	公共库、Federation状态存储
hadoop-yarn-server-applicationhistoryservice	AHS	65	应用历史记录与Timeline V1
hadoop-yarn-server-timelinehistoryservice	TimelineService V2	64	V2版本Timeline存储
hadoop-yarn-server-timelinehistoryservice-hbase-common	HBase Common	50+	HBase存储公共模块
hadoop-yarn-server-timelinehistoryservice-hbase-client	HBase Client	20+	HBase客户端读写
hadoop-yarn-server-timelinehistoryservice-hbase-server	HBase Server	15+	HBase协处理器
hadoop-yarn-server-timelinehistoryservice-documentstore	DocStore	10+	文档型存储后端
hadoop-yarn-server-router	Router	67	Federation路由层
hadoop-yarn-server-globalpolicygenerator	GPG	28	全局联邦策略生成器
hadoop-yarn-server-sharedcache	SCM	19	共享缓存管理服务

模块名称	简称	文件数	核心职责
hadoop-yarn-server-web-proxy	WebProxy	13	AM Web代理
hadoop-yarn-server-tests	Tests	—	集成测试模块

表 1-1: hadoop-yarn-server 子模块总览

1.2 服务入口点

YARN Server 共有 8 个可独立启动的服务进程，每个进程通过 `main()` 方法作为入口启动：

- `ResourceManager`: 集群的全局资源管理器，负责接收应用提交、资源调度、节点管理
- `NodeManager`: 节点代理，负责容器启动/停止/监控、资源隔离
- `ApplicationHistoryServer`: 历史应用记录查询服务，Timeline Service V1
- `TimelineReaderServer`: Timeline Service V2 读取服务
- `GlobalPolicyGenerator`: Federation 全局策略生成器
- `Router`: Federation 路由网关，客户端透明访问多集群
- `SharedCacheManager`: 分布式缓存管理服务
- `WebAppProxyServer`: ApplicationMaster Web UI 安全代理

2. 整体架构设计

YARN Server 采用经典的 Master/Slave 架构。ResourceManager 作为 Master 节点，负责全局资源管理和调度决策；NodeManager 作为 Slave 节点，运行在每台工作机器上，负责具体的容器管理和资源监控。在 Federation 场景下，Router 作为统一入口代理多个 YARN 集群的请求。

2.1 核心架构图

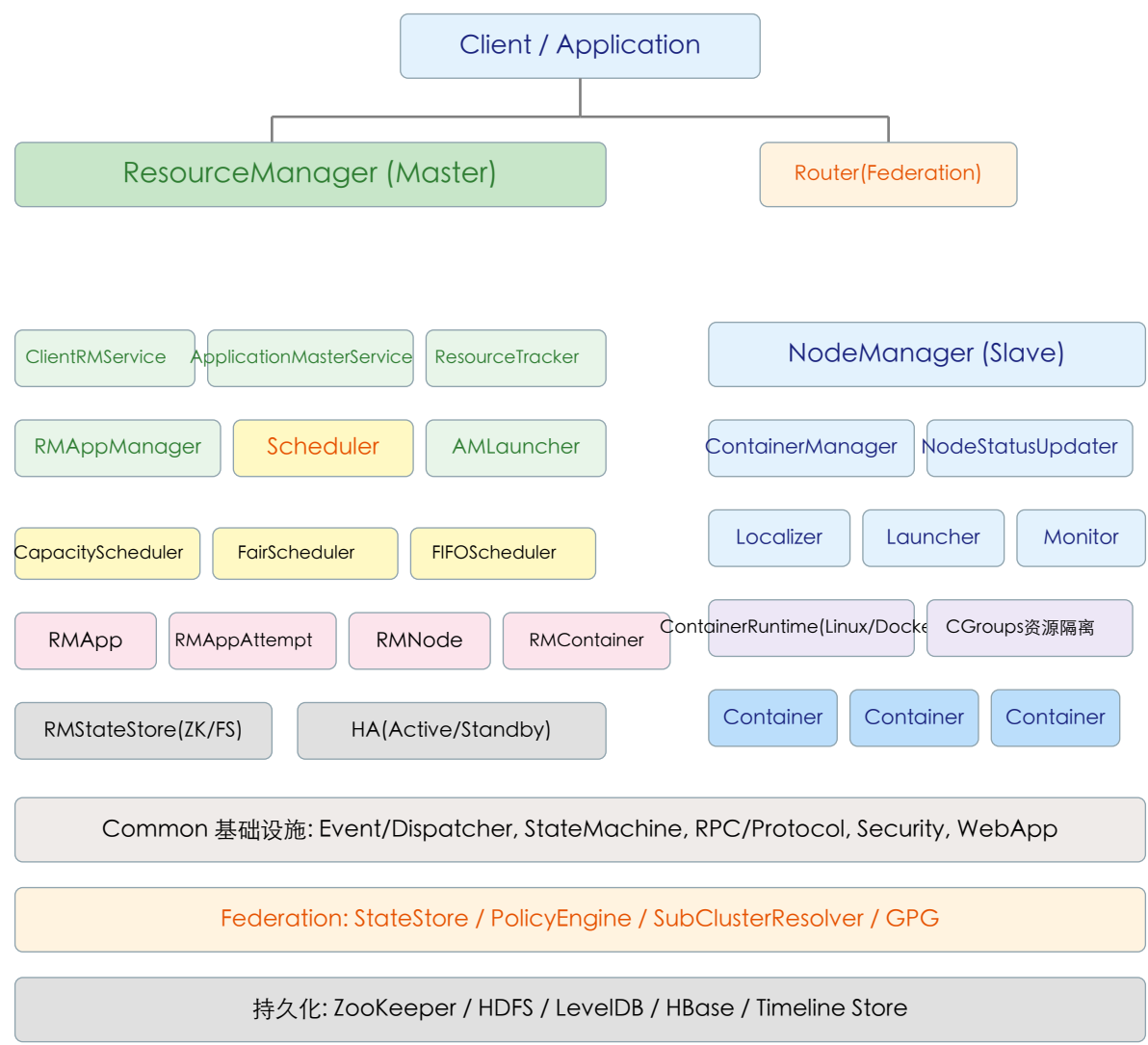


图 2-1: YARN Server 整体架构图

2.2 分层架构说明

YARN Server 的架构可以从上到下分为以下几层：

层次	核心组件	职责
客户端接入层	ClientRMService, AMRMService	接收客户端和AM的RPC请求
应用管理层	RMAppManager, RMApp状态机	应用提交、生命周期管理、Attempt管理
资源调度层	CapacityScheduler, FairScheduler	队列管理、资源分配、抢占
节点管理层	ResourceTracker, RMNode状态机	节点注册/心跳/去委任
容器管理层	RMContainer, ContainerManagerImpl	容器分配/启动/监控/回收
联邦路由层	Router, FederationClientInterceptor	跨集群请求路由和聚合
事件驱动层	AsyncDispatcher, EventHandler	异步事件分发总线
状态持久化层	RMStateStore, NMStateStore	ZK/FS/LevelDB状态恢复

表 2-1: YARN Server 分层架构

3. ResourceManager 核心架构

ResourceManager 是 YARN 集群的大脑，继承自 CompositeService，采用组合服务模式（Composite Pattern）管理内部所有子服务。RM 的服务分为两类：Always-On 服务（无论 HA 状态如何都运行）和 Active 服务（仅在 Active RM 中运行）。

3.1 RM 服务组件

ResourceManager 内部包含以下核心组件，它们共同构成了一个完整的资源管理体系：

组件名称	职责描述	服务类型
ClientRMService	处理客户端RPC请求（提交应用、查询状态、Kill应用等）	Always-On
ApplicationMasterService	处理AM的RPC请求（资源申请、心跳、注册/注销）	Active
ResourceTrackerService	处理NM的RPC请求（注册、心跳）	Active
ResourceScheduler	资源调度器（Capacity/Fair/FIFO）	Active
RMAppManager	应用管理器，负责应用创建和移除	Active
ApplicationMasterLauncher	AM启动器，向NM发送启动AM容器命令	Active
NMLivelinessMonitor	NM存活监控，定期检测NM心跳超时	Active
AMLivelinessMonitor	AM存活监控，定期检测AM心跳超时	Active
RMStateStore	状态持久化（ZK/FS），支持RM故障恢复	Active
AdminService	管理员操作接口（刷新队列/节点列表等）	Always-On
RMNodeLabelsManager	节点标签管理	Active
ContainerAllocationExpirer	容器分配过期监控	Active
DelegationTokenRenewer	安全Token续期服务	Active
FederationStateStoreService	Federation状态存储服务	Active

表 3-1: ResourceManager 核心组件

3.2 RM 事件驱动架构

RM 内部采用中央事件分发器（AsyncDispatcher）实现组件间的松耦合通信。每种事件类型注册一个EventHandler，Dispatcher 通过事件类型将事件路由到对应的 Handler。核心事件类型包括：

事件类型	描述	关键事件
RMAppEventType	应用级事件	START, KILL, RECOVER, APP_REJECTED, ATTEMPT_*
RMAppAttemptEventTy e	应用尝试事件	START, KILL, REGISTERED, CONTAINER_ALLOCATED, EXPIRE
RMNodeEventType	节点事件	STARTED, STATUS_UPDATE, EXPIRE, DECOMMISSION, RECONNECTED
SchedulerEventType	调度事件	NODE_ADDED/REMOVED, APP_ADDED/REMOVED, NODE_UPDATE
RMContainerEventType	容器事件	START, ACQUIRED, LAUNCHED, FINISHED, KILL, EXPIRE
AMLauncherEventType	AM启动事件	LAUNCH, CLEANUP
NodesListManagerEventT ype	节点列表事件	NODE_USABLE, NODE_UNUSABLE

表 3-2: RM 核心事件类型

3.3 RM 高可用架构

ResourceManager 支持 Active/Standby 高可用模式。核心机制包括：

- Leader Election: 基于 ZooKeeper（CuratorBasedElectorService 或 ActiveStandbyElectorBasedElectorService）进行主节点选举
- RMStateStore: 支持 ZKRMStateStore（ZooKeeper）和 FileSystemRMStateStore（HDFS）两种持久化后端，用于保存应用/Attempt 状态、安全Token等
- Work-Preserving Recovery: Active RM 故障切换后，Standby RM 从 StateStore 恢复应用状态，NM 重新连接后 Container 继续运行
- RMActiveServices: 作为 CompositeService 封装所有仅在 Active RM 上运行的服务，切换时统一启停

4. NodeManager 核心架构

NodeManager 继承自 CompositeService 并实现 EventHandler<NodeManagerEvent> 接口，运行在每个集群节点上，负责容器生命周期管理、本地资源管理和状态上报。

4.1 NM 核心组件

组件名称	角色	核心职责
ContainerManagerImpl	容器管理核心，实现ContainerManagementProtocol	接收/处理AM的启动/停止容器请求
NodeStatusUpdater(Impl)	节点状态上报器	定期向RM发送心跳，报告节点资源和容器状态
ResourceLocalizationService	资源本地化服务	从HDFS下载容器所需的JAR/文件到本地
ContainersLauncher	容器启动器	通过ContainerExecutor执行容器的实际启动命令
ContainersMonitorImpl	容器资源监控	监控容器CPU/内存使用，超限则Kill
ContainerScheduler	本地容器调度	管理Guaranteed/Opportunistic容器的本地队列
LogAggregationService	日志聚合服务	将容器日志上传到HDFS集中存储
DeletionService	文件清理服务	异步清理过期的本地化资源和容器工作目录
NodeHealthCheckerService	节点健康检查	检测节点磁盘健康状态
NMStateStoreService	NM状态持久化	基于LevelDB，支持NM重启恢复容器状态

表 4-1: NodeManager 核心组件

4.2 NM 容器执行引擎

NodeManager 支持多种容器运行时，通过 ContainerExecutor 抽象层实现可插拔：

- DefaultContainerExecutor: 默认执行器，以NM进程的用户身份启动容器
- LinuxContainerExecutor: Linux 原生执行器，支持 CGroups v1/v2 资源隔离
- DockerLinuxContainerRuntime: Docker 容器运行时，在Docker中执行任务
- RuncContainerRuntime: 基于 Runc 的轻量级容器运行时

资源隔离通过 CGroupsHandler (v1) 和 CGroupsV2Handler (v2) 实现，支持 CPU、内存、网络等资源维度的限制。

5. 状态机设计模式详解

YARN RM 广泛采用 StateMachineFactory 实现有限状态机（FSM），这是整个系统最核心的设计模式。每个状态机通过声明式方式定义状态、事件和转移函数的映射关系，配合读写锁保证线程安全。

5.1 RMAp 状态机

RMAp 表示一个提交到 YARN 的应用，其状态机管理应用从提交到完成的全生命周期。核心状态包括：NEW → NEW_SAVING → SUBMITTED → ACCEPTED → RUNNING → FINISHING → FINISHED，以及 KILLING → KILLED、FAILED 等终态。

状态	含义	主要转移
NEW	应用刚创建	接收START事件后进入NEW_SAVING
NEW_SAVING	持久化应用状态到StateStore	保存成功后进入SUBMITTED
SUBMITTED	已提交到调度器	调度器接受后进入ACCEPTED
ACCEPTED	调度器已接受，等待AM启动	AM注册后进入RUNNING
RUNNING	AM已注册，应用正在运行	AM注销或完成后进入FINISHING/FINISHED
FINISHING	AM已注销，等待最终清理	清理完成后进入FINISHED
FINAL_SAVING	保存最终状态到StateStore	保存完成后转入终态
FINISHED	应用正常完成（终态）	—
FAILED	应用失败（终态）	—
KILLED	应用被Kill（终态）	—
KILLING	正在Kill应用的Attempt	Attempt被Kill后进入FINAL_SAVING

表 5-1: RMAp 状态机状态表

5.2 RMApAttempt 状态机

每个RMAp可以有多个RMApAttempt（应用尝试），代表一次AM的执行。状态机管理从启动到完成的全过程。

状态	含义	关键转移
NEW	尝试刚创建	START → SUBMITTED

状态	含义	关键转移
SUBMITTED	已提交到调度器	ATTEMPT_ADDED → SCHEDULED
SCHEDULED	等待AM容器分配	CONTAINER_ALLOCATED → ALLOCATED_SAVING
ALLOCATED_SAVING	保存AM容器信息	ATTEMPT_NEW_SAVED → ALLOCATED
ALLOCATED	AM容器已分配	LAUNCHED → LAUNCHED
LAUNCHED	AM已启动，等待注册	REGISTERED → RUNNING
RUNNING	AM正在运行	UNREGISTERED → FINAL_SAVING
FINAL_SAVING	保存最终状态	转入终态(FINISHED/FAILED/KILLED)

表 5-2: RMAppAttempt 状态机状态表

5.3 RMNode 状态机

RMNode 表示集群中的一个工作节点，状态机管理节点从注册到退出的全生命周期。

状态	含义	关键转移
NEW	节点刚注册	STARTED → RUNNING/UNHEALTHY
RUNNING	节点正常运行	STATUS_UPDATE保持或→UNHEALTHY; EXPIRE→LOST
UNHEALTHY	节点不健康	STATUS_UPDATE恢复→RUNNING; EXPIRE→LOST
DECOMMISSIONING	优雅退役中	容器运行完毕后→DECOMMISSIONED
DECOMMISSIONED	已退役（终态）	—
LOST	心跳超时丢失（终态）	—
REBOOTED	节点重启（终态）	—
SHUTDOWN	节点关闭（终态）	—

表 5-3: RMNode 状态机状态表

5.4 RMContainer 状态机

RMContainer 是RM端对容器的抽象，跟踪容器从分配到完成的状态变化：

状态	含义	关键转移
NEW	容器刚创建	START→ALLOCATED; RESERVED→RESERVED; RECOVER→RUNNING/COMPLETED
RESERVED	资源已预留	START→ALLOCATED; KILL→KILLED
ALLOCATED	已分配，等待AM获取	ACQUIRED→ACQUIRED; EXPIRE→EXPIRED
ACQUIRED	AM已获取容器	LAUNCHED→RUNNING; FINISHED→COMPLETED
RUNNING	容器正在运行	FINISHED→COMPLETED; KILL→KILLED; RELEASED→RELEASED
COMPLETED	正常完成（终态）	—
EXPIRED	超时未获取（终态）	—
KILLED	被Kill（终态）	—
RELEASED	被释放（终态）	—

表 5-4: RMContainer 状态机状态表

6. 调度器体系架构

调度器是YARN的核心大脑，决定如何将有限的集群资源分配给各个应用。YARN支持可插拔的调度器，通过配置 `yarn.resourcemanager.scheduler.class` 切换实现。

6.1 调度器继承体系

调度器的类层次结构如下：

```
ResourceScheduler (■■)
■■ AbstractYarnScheduler<T, N> (■■■■■)
■■ CapacityScheduler (■■■■■ - ■■ & ■■■■)
■■ FairScheduler (■■■■■)
■■ FifoScheduler (FIFO■■■■)
```

6.2 AbstractYarnScheduler 基类

`AbstractYarnScheduler` 是所有调度器的抽象基类，继承 `AbstractService` 并实现 `ResourceScheduler` 接口。它提供了以下核心能力：

- `ClusterNodeTracker`: 追踪集群中所有节点的资源状态
- 读写锁机制: `ReentrantReadWriteLock` 保护队列变更、应用增删、容器分配等操作
- `SchedulingMonitorManager`: 调度监控管理（抢占监视器等）
- `UpdateThread`: 异步更新线程，定期触发调度决策
- 应用管理: `ConcurrentMap<ApplicationId, SchedulerApplication>` 维护所有应用状态

6.3 CapacityScheduler (容量调度器)

`CapacityScheduler` 是 YARN 默认且生产环境推荐使用的调度器（134KB+ 代码），核心特性包括：

特性	说明
层次化队列	树形队列结构，支持 <code>ParentQueue</code> 和 <code>LeafQueue</code> ，队列容量可配置百分比
容量保证	每个队列保证最小容量，空闲时可弹性使用其他队列资源
多租户隔离	队列级别的ACL访问控制，按用户/组配置权限
抢占机制	<code>PreemptionManager</code> 支持按队列容量比例回收超额使用的资源
节点标签	<code>Node Labels/Partition</code> 实现异构集群资源分区调度
放置约束	<code>Placement Constraints</code> 支持反亲和性等高级调度约束
异步调度	<code>AsyncSchedulingConfiguration</code> 支持多线程异步调度

特性	说明
可变配置	MutableCSConfigurationProvider 支持运行时动态修改队列配置
应用优先级	AppPriorityACLsManager 管理应用优先级
最大运行应用数	CSMaxRunningAppsEnforcer 控制每个队列/用户的并发应用数

表 6-1: CapacityScheduler 核心特性

6.4 调度核心流程

CapacityScheduler 的资源分配发生在 NodeUpdate 事件处理中：

```
1. NodeManager  ■■■■ → ResourceTracker  ■■
2.  ■■  NODE_UPDATE SchedulerEvent
3. CapacityScheduler.nodeUpdate()  ■■■
4.  ■■■■■■■■: RootQueue.assignContainers()
5. ParentQueue  ■■■■■■■■■■
6. LeafQueue  ■■■■■■■■■■
7. FiCaSchedulerApp.assignContainers()  ■■■■■■
8.  ■■ ResourceCommitRequest  ■■■■■■
9.  ■■ RMContainer/SchedulerNode  ■■
```


代理发送请求。对于聚合类请求（如 `getApplications`），会并行向所有 `SubCluster` 发送请求并合并结果。

8. 核心流程 — 应用提交与执行

应用提交是 YARN 最核心的流程之一，涉及 Client、RM、Scheduler、NM 等多个组件的协作。以下是完整的应用提交到执行的时序流程：

8.1 应用提交时序



图 8-1: 应用提交时序（文本流程图）

8.2 关键参与类

类名	角色	关键行为
ClientRMService	接收submitApplication RPC	参数校验、权限检查、转发给RAppManager
RAppManager	应用管理入口	创建RAppImpl，触发START事件
RAppImpl	应用状态机	驱动应用经历NEW→SUBMITTED→ACCEPTED→RUNNING
RAppAttemptImpl	Attempt状态机	管理AM容器分配→启动→注册→运行

类名	角色	关键行为
ResourceScheduler	资源调度器	接收APP_ADDED/ATTEMPT_ADDED事件，分配AM容器
ApplicationMasterLauncher	AM启动器	创建AMLauncherEvent，通过RPC向NM启动AM容器

表 8-1: 应用提交流程关键参与类

9. 核心流程 — 容器生命周期管理

容器 (Container) 是 YARN 中资源分配的基本单位。容器的生命周期涉及 RM 端 (RMContainer) 和 NM 端 (ContainerImpl) 两个状态机的协同。

9.1 容器分配流程

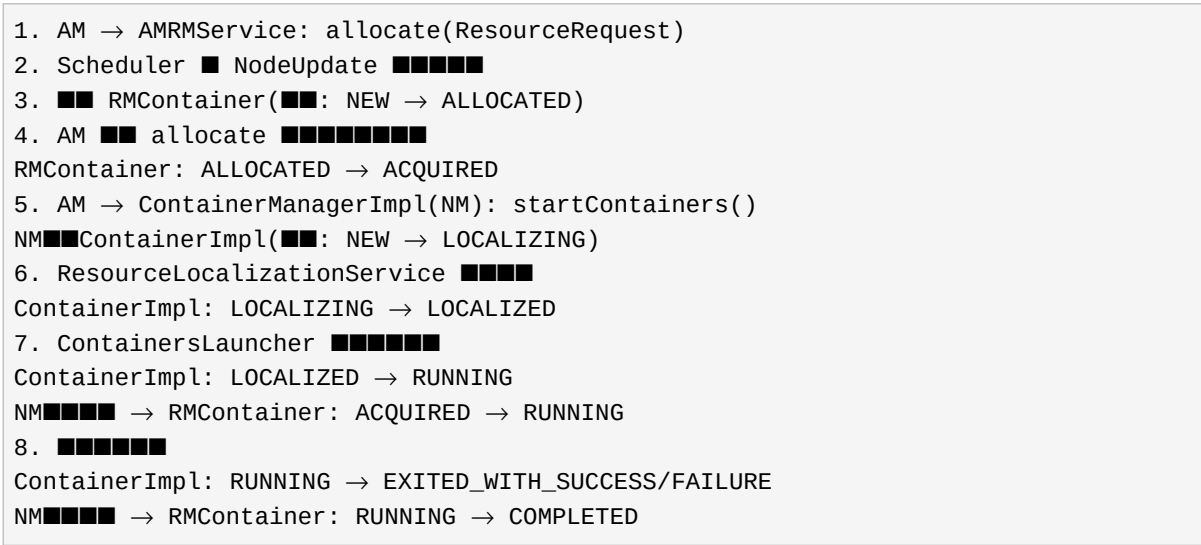


图 9-1: 容器完整生命周期

9.2 NM端容器状态机

NM端的 ContainerImpl 有更细粒度的状态管理，反映了资源本地化、进程启动、监控等阶段：

状态	描述
NEW	容器刚创建
LOCALIZING	正在下载资源到本地
LOCALIZED	资源下载完成，等待启动
SCHEDULED	ContainerScheduler已调度
RUNNING	容器进程正在运行
REINITIALIZING	容器重初始化中
RELAUNCHING	容器重启中
EXITED_WITH_SUCCESS	正常退出
EXITED_WITH_FAILURE	异常退出

状态	描述
KILLING	正在Kill
CONTAINER_CLEANEDUP_AFTER_KILL	Kill后清理完成
DONE	最终完成

表 9-1: NM 端 ContainerImpl 状态

10. 核心流程 — 节点心跳与资源更新

节点心跳是 RM 和 NM 之间的核心通信机制，承载了资源状态上报、容器状态同步、命令下发等关键功能。

10.1 心跳流程

```
NM(NodeStatusUpdater) → RM(ResourceTrackerService): nodeHeartbeat()  
■■ ■■■■:  
■ ■■ NodeStatus (■■■■■■/■■■■■■)  
■ ■■ ContainerStatus[] (■■■■/■■■■■■■■)  
■ ■■ keepAliveApplications (■■■■■■)  
■ ■■ LogAggregationReport (■■■■■■)  
■  
■■ RM■■:  
■ ■■ ■■ NMLivelinessMonitor (■■■■■■■■)  
■ ■■ ■■ RMNodeEvent(STATUS_UPDATE)  
■ ■ ■■ RMNode■■■: StatusUpdateWhenHealthyTransition  
■ ■ ■■ ■■■■■■■■ nodeUpdateQueue  
■ ■ ■■ ■■ NodeUpdateSchedulerEvent  
■ ■ ■■ Scheduler.nodeUpdate() → ■■■■!  
■ ■■ ■■■■■■■■■■  
■  
■■ ■■■■ (NodeHeartbeatResponse):  
■■ containersToCleanup (■■■■■■■■)  
■■ applicationsToCleanup (■■■■■■■■)  
■■ containersToSignal (■■■■■■)  
■■ containersToBeRemovedFromNM (■NM■■■■■■■■)  
■■ nextHeartBeatInterval (■■■■■■■■)
```

图 10-1: 节点心跳流程

10.2 心跳驱动调度

值得注意的是，CapacityScheduler 的资源分配是心跳驱动的：每次 NM 心跳触发一次 nodeUpdate()，调度器在此事件中尝试为待调度的应用分配该节点上的可用资源。这意味着节点越多，心跳越频繁，调度吞吐量越高。对于大规模集群，可启用异步调度模式（asyncSchedulingConf）通过独立线程持续执行调度，不受心跳频率限制。

12. 关键设计模式总结

YARN

Server

源码中大量运用了经典的设计模式，这些模式共同构建了一个高度模块化、可扩展的系统。

设计模式	应用位置	说明
事件驱动/观察者模式	AsyncDispatcher + EventHandler	所有组件通过事件通信，解耦发送者和接收者。Dispatcher作为中央事件总线，各组件注册EventHandler处理特定类型事件
状态机模式	StateMachineFactory	RMAApp/RMAAppAttempt/RMNode/RMContainer/NMContainer 均使用声明式状态机，通过 addTransition() 定义状态×事件→新状态×动作的完整映射
组合服务模式	CompositeService	ResourceManager/NodeManager 继承 CompositeService，管理多个子Service的init/start/stop生命周期
责任链模式	Interceptor Chain (Router)	Router模块的ClientRequestInterceptor链式处理请求，支持灵活插入日志/鉴权/限流/路由等逻辑
工厂模式	createScheduler() / createContainerExecutor()	通过反射+配置动态创建调度器、容器执行器等核心组件，支持运行时可插拔
策略模式	PlacementRule / FederationPolicy	队列放置规则和联邦路由策略均可配置替换，实现不同的调度/路由策略
模板方法模式	AbstractYarnScheduler	定义调度流程骨架，子类(CapacityScheduler/FairScheduler)重写具体步骤
代理模式	WebAppProxy / FederationProxy	AM Web UI 代理和 Federation SubCluster RPC 代理
单例模式	ClusterMetrics / RouterMetrics	指标收集器使用单例，全局唯一实例统一收集集群/路由指标
读写锁模式	ReentrantReadWriteLock	状态机操作使用ReadLock/WriteLock，读操作并发，写操作互斥，平衡性能与安全

表 12-1: YARN Server 核心设计模式

13. 关键场景调用链分析

13.1 应用提交调用链

```
Client.submitApplication()  
→ ClientRMService.submitApplication()  
→ RMAppManager.submitApplication()  
→ new RMAppImpl() // ■■■■■■  
→ rmContext.getDispatcher().getEventHandler()  
.handle(new RMAppEvent(appId, START))  
→ RMAppImpl.RMAppNewlySavingTransition.transition()  
→ rmContext.getStateStore().storeNewApplication()  
→ [StateStore callback] RMAppEvent(APP_NEW_SAVED)  
→ AddApplicationToSchedulerTransition.transition()  
→ scheduler.handle(AppAddedSchedulerEvent)  
→ CapacityScheduler.addApplication()  
→ LeafQueue.submitApplication()  
→ [Scheduler callback] RMAppEvent(APP_ACCEPTED)  
→ StartAppAttemptTransition.transition()  
→ new RMAppAttemptImpl()  
→ RMAppStartAttemptEvent
```

13.2 AM容器启动调用链

```
Scheduler.allocateContainersToNode() // AM■■■■■  
→ RMContainerEvent(START) → RMContainer: NEW→ALLOCATED  
→ RMAppAttemptEvent(CONTAINER_ALLOCATED)  
→ AMContainerAllocatedTransition.transition()  
→ RMStateStore.storeNewApplicationAttempt()  
→ [callback] RMAppAttemptEvent(ATTEMPT_NEW_SAVED)  
→ AMLauncher.handle(AMLauncherEvent(LAUNCH))  
→ AMLauncher.launch() // ■■■■  
→ ContainerManagementProtocol.startContainers()  
→ NM.ContainerManagerImpl.startContainers()  
→ ContainerImpl■■■■: NEW → LOCALIZING  
→ ResourceLocalizationService (■■■■)  
→ LOCALIZING → LOCALIZED → RUNNING
```

13.3 心跳触发调度调用链

```
NM.NodeStatusUpdater → RM.ResourceTrackerService.nodeHeartbeat()  
→ NMLivelinessMonitor.receivedPing(nodeId)  
→ Dispatcher.handle(RMNodeEvent(nodeId, STATUS_UPDATE))  
→ RMNodeImpl.StatusUpdateWhenHealthyTransition.transition()  
→ ■■■containerStatuses, ■■■nodeUpdateQueue  
→ Dispatcher.handle(NodeUpdateSchedulerEvent)  
→ CapacityScheduler.nodeUpdate(RMNode)  
→ processCompletedContainers()
```



```
→ allocateContainersToNode(FiCaSchedulerNode)
→ RootQueue.assignContainers()
→ ParentQueue■■■■■ → LeafQueue
→ FiCaSchedulerApp.assignContainers()
→ ■■ResourceRequest → ■■RMContainer
```

13.4 容器完成调用链

```
NM: Container■■■■■
→ ContainerImpl: RUNNING → EXITED_WITH_SUCCESS/FAILURE
→ NM■■■■■ ContainerStatus(COMplete)
→ RM.ResourceTrackerService.nodeHeartbeat()
→ RMNodeImpl■■■completedContainerStatuses
→ Scheduler.completedContainer(RMContainer)
→ RMContainerEvent(FINISHED) → RUNNING → COMPLETED
→ ■■■■■■■■■■■■
→ ■■■■■■■■■■
→ RMAppAttemptEvent(CONTAINER_FINISHED)
→ ■■■■AM■■: RMAppAttempt■■■■■AM■■
→ ■■■■■■■■: ■■■■■■■■
```

13.5 Federation请求路由调用链

```
Client → Router.ClientRMSERVICE
→ RouterClientRMSERVICE.submitApplication()
→ ClientRequestInterceptor█:
→ [█████████...]
→ FederationClientInterceptor.submitApplication()
→ policyFacade.getHomeSubcluster() // ██████SubCluster
→ getClientRMPProxyForSubCluster(subClusterId)
→ proxy.submitApplication() // ██████RMSERVICE
→ federationFacade.addApplicationHomeSubCluster()
// ███App████StateStore
→ return response
```

13.6 RM故障恢复调用链

```

StandbyRM ■■■■Active:
→ ResourceManager.transitionToActive()
→ RMActiveServices.serviceStart()
→ RMStateStore.loadState()
→ ■■■■■■■■■■Application/Attempt/SecurityToken
→ RMAAppManager.recover(RMState)
→ ■■■■Application:
→ new RMAAppImpl(recovered=true)
→ RMAAppEvent(RECOVER) → RMAAppRecoveredTransition
→ ■■■■■■■■■■■■■■■■■■■■
→ ■■RMAAppAttempt
→ ■■NM■■■■■:

```

```
→ NM.nodeHeartbeat() → RECONNECTED■■■  
→ RMNodeImpl■■■■■  
→ ■■■Container■■■ (Work-Preserving)
```

14. 接口与扩展点

YARN Server 通过精心设计的接口体系提供了丰富的扩展点，允许用户自定义调度策略、容器执行引擎、安全认证等核心行为。

接口/扩展点	用途	扩展方式
ResourceScheduler	资源调度器接口	自定义调度算法，继承AbstractYarnScheduler
ContainerExecutor	容器执行器	自定义容器启动方式（Docker/Runc/自定义）
PlacementRule	队列放置规则	自定义应用到队列的映射逻辑
ClientRequestInterceptor	客户端请求拦截器(Router)	自定义请求处理逻辑（限流/鉴权）
RMAdminRequestInterceptor	管理请求拦截器(Router)	自定义管理操作拦截
RESTRequestInterceptor	REST请求拦截器(Router)	自定义REST API处理
ConfigurationProvider	配置提供者	自定义配置加载方式
RMStateStore	状态持久化	自定义状态存储后端
NMStateStoreService	NM状态持久化	自定义NM状态恢复机制
NodeLabelsProvider	节点标签提供者	自定义节点标签获取方式
ContainerStateTransitionListener	容器状态转移监听器	自定义容器状态变化处理逻辑
SystemMetricsPublisher	系统指标发布者	自定义指标发布到Timeline Service
FederationPolicy	联邦路由策略	自定义SubCluster选择算法
AuxiliaryService	辅助服务(NM)	在NM上运行自定义服务（如Shuffle Service）
ContainerRuntime	容器运行时	自定义容器进程隔离和执行方式

表 14-1: YARN Server 核心接口与扩展点

15. 总结

本文档对 Hadoop YARN Server (release-3.3.5-RC0) 的核心架构进行了全面深入的分析。以下是关键发现和架构特点总结：

15.1 架构特点

- 事件驱动架构: 整个系统基于 AsyncDispatcher + EventHandler 的事件驱动模型，实现了组件间的高度解耦。所有核心操作（应用提交、容器分配、节点管理）都通过事件异步触发。
- 状态机驱动: 四大核心实体（RMApp、RMAppAttempt、RMNode、RMContainer）均采用声明式状态机管理生命周期，状态转移逻辑清晰可维护。
- 可插拔设计: 调度器、容器执行器、状态存储等核心组件均通过接口抽象，支持通过配置动态切换实现。
- 组合服务模式: ResourceManager 和 NodeManager 使用 CompositeService 管理子服务生命周期，服务启停有序。
- Federation 可扩展: 通过 Router + GPG + FederationStateStore 支持多集群联邦，拦截器链模式使请求处理逻辑可灵活扩展。
- HA 容错: RM 支持 Active/Standby 高可用，基于 ZK 选主，Work-Preserving Recovery 保证容器在 RM 故障切换后继续运行。

15.2 规模数据

指标	数值
模块总数	14 个子模块
源代码文件	1165+ Java 文件
最大单文件	FederationInterceptorREST.java (164 KB)
核心状态机	4 个 (RMApp/RMAppAttempt/RMNode/RMContainer) + NM端容器状态机
调度器实现	3 种 (CapacityScheduler/FairScheduler/FifoScheduler)
容器运行时	3 种 (Default/Linux+Docker/Runc)
核心接口	15+ 个可扩展接口
服务入口	8 个可独立启动的进程
事件类型	7+ 种核心事件类型

表 15-1: YARN Server 规模数据

— 文档结束 —